SOFTWARE REQUIREMENTS SPECIFICATION

SettleUp

A Group Expense Splitter with Greedy Debt Minimization

Project Type: Frontend Web Development Project **Technology:** HTML5, CSS3, Vanilla JavaScript, Browser localStorage
Course: B.Tech, Semester III **Prepared by:** Nikhil Varshney **Roll Number:** 22515000089 **Project Guide:** Gautam Mukherjee **Institution:** GLA University, Greater Noida

Document Version: 2.3 **Date:** 15 August 2026 **Document Status:** Baseline

Revision History

Version	Date	Author	Description of Change
0.1	30 July 2026	Nikhil Varshney	Initial draft — project idea, scope outline
1.0	07 August 2026	Nikhil Varshney	First complete draft with functional and non-functional requirements
2.0	14 August 2026	Nikhil Varshney	Baseline. Removed unjustified numeric targets and unplanned features from v1.0. Added Business Rules, Algorithmic Requirements, Testing, Limitations and Future Enhancements. Corrected the description of the transaction-count benefit, which v1.0 conflated with algorithmic time complexity.
2.1	14 August 2026	Nikhil Varshney	Consistency review. Corrected eight business-rule cross-references in Section 3 that pointed at the wrong rules, and one reference to a requirement that did not exist. Moved recommended features out of the mandatory FR series into Section 3.14 under RF identifiers. Replaced the three-layer architecture description with the project's actual file organisation. Made the amount representation consistent between calculation, storage and export. Reworded three requirements that could not be verified as written. No planned requirement was removed; requirements were reorganised, renumbered and clarified.
2.2	14 August 2026	Nikhil Varshney	Second consistency review. Resolved a contradiction between FR-01, which rejected duplicate member names, and Section 5.4.1, whose justification for identifier-based references rests on duplicate names being possible; duplicates are now permitted and the interface distinguishes them. Recorded the group question as an open decision (OD-01) instead of leaving it implied. Removed "best" and "optimal" framing from Section 7.4. Restated the n(n−1)/2 figure as a bound on member pairs rather than a count of payments. Separated the design decision inside FR-11 from its requirement. Reduced browser and device claims to those that will actually be tested. Made file names indicative rather than prescribed.
2.3	15 August 2026	Nikhil Varshney	Closed the open decision on group handling: one implicit group is now the settled position for this version, and Section 2.8 has been removed. Moved the balance derivation strategy out of FR-11 into a separate design decision, DD-01. Removed the exception clause from constraint C-03, which now prohibits third-party JavaScript libraries without qualification. Extended the OBJ-04 traceability row to the persistence requirements.

Table of Contents

---

1. INTRODUCTION

1.1 Purpose

This document specifies the requirements for **SettleUp**, a browser-based application that records shared expenses within a group and computes a settlement plan showing who should pay whom, and how much.

The document is written for three readers:

Reader	What they need from this document
Project guide and evaluating faculty	A clear statement of what is being built, on what basis it will be judged complete, and what has deliberately been left out
The developer (the author)	A specification precise enough to build from, and a record of decisions already taken
Any future developer	Enough context to extend the system without re-deriving the design

This document states **what** the system must do. It does not prescribe **how** the code should be organised, beyond the technology constraints recorded in Section 2.5.

1.2 Scope

SettleUp is a single-user, client-side web application. It runs entirely inside a web browser. It has no server component and no database server. Once the page and its files have loaded, the application requires no network communication for any of its functions.

The problem it addresses is a familiar one. When a group of people share costs — during a trip, in shared accommodation, or over a series of group meals — different members pay different amounts at different times. Reconciling this afterwards is done manually, usually on a calculator, and is prone to two kinds of error: arithmetic mistakes in computing each person's share, and unnecessary payments when money is passed through a member who does not need to hold it.

SettleUp addresses both. It computes shares and balances arithmetically, and it reduces the resulting debts to a small set of payments before presenting them.

In scope

Area	Included
Group	One group, held implicitly — see the note below this table. No group entity, no group name.
Members	Add and display members; remove members subject to the rules in Section 8.1
Expenses	Record expenses with description, amount, payer, category and participants
Splitting	Equal, exact-amount and percentage splits
Balances	Net balance per member; classification into creditors and debtors
Settlement	A payment list produced by greedy debt minimization
History	Chronological expense list with search and filtering
Analysis	Category-wise spending totals
Persistence	Automatic save and restore using browser localStorage
Portability	Export to and import from a JSON file

On the notion of a group. This version manages exactly one group, and does so implicitly: the members and expenses held by the application *are* the group. There is no `Group` entity in the data model, no group name, and no creation step — the user opens the application and begins adding members.

This is a decision, not an omission. Introducing a named group would add a level to the data model, a screen to the interface, and a set of requirements around creating, renaming and switching between groups, none of which changes what the application actually computes. Managing several groups is recorded as a future enhancement in Section 11; until then, a second trip is handled by exporting the first (FR-19) and starting fresh.

Out of scope

The following are explicitly **not** part of this project. They are listed so that their absence is understood as a decision, not an omission.

Excluded	Reason
Actual money transfer or payment processing	The application produces instructions; the payments themselves happen outside it
User accounts, login, authentication	No server exists to authenticate against
Multi-device or multi-user synchronisation	Data is stored in one browser on one device
Backend server or database	Frontend-only project by academic requirement
External APIs or third-party services	Not required by any feature
Multiple simultaneous groups	Out of scope for this version; see Section 11
Multi-currency support	All amounts within the application are assumed to be in a single currency

1.3 Project Objectives

ID	Objective
OBJ-01	Remove manual arithmetic from the reconciliation of shared group expenses
OBJ-02	Reduce the number of payments needed to settle a group, so that members make fewer transfers than a naive pairwise settlement would require
OBJ-03	Provide complete functionality without requiring an account, an internet connection, or any installation
OBJ-04	Keep all of the group's financial data on the user's own device
OBJ-05	Demonstrate practical competence in HTML, CSS and JavaScript, including DOM manipulation, form handling, state management, and the implementation of a non-trivial algorithm without external libraries

1.3.1 Distinction between Scope and Objectives

These two terms are often confused, and the distinction is worth stating.

Scope describes the *boundary* of the system — the set of things it will and will not do. It answers "what is inside this project?"

Objective describes the *purpose* — what is achieved by building it. It answers "why is this project worth doing?"

For example, "the system shall generate a settlement plan" is scope. "Reduce the number of payments members must make" is an objective. The first is a feature; the second is the reason the feature exists.

1.4 Definitions, Acronyms and Abbreviations

Term	Definition
Member	A person participating in the group, identified internally by a unique numeric ID

Term	Definition
Expense	A single recorded cost, with an amount, a payer, a category, and a set of participants
Payer	The member who paid for a given expense
Participant	A member among whom a given expense is divided
Share	The portion of one expense attributable to one participant
Split method	The rule used to divide an expense: equal, exact, or percentage
Net balance	For a member: total amount paid, minus the total of their shares across all expenses
Creditor	A member whose net balance is positive — money is owed *to* them
Debtor	A member whose net balance is negative — they owe money
Settled member	A member whose net balance is zero
Settlement	A list of payments which, if made, brings every net balance to zero
Debt minimization	Reducing the number of payments in a settlement without changing any member's net position
Greedy algorithm	An algorithm that makes the locally best choice at each step, without reconsidering earlier choices
Pairwise settlement	A naive approach in which every debtor pays every creditor they owe, separately
DOM	Document Object Model — the browser's live, in-memory representation of the page
localStorage	A browser feature that stores string data on the user's device, persisting across sessions
JSON	JavaScript Object Notation — a plain-text format for structured data
Vanilla JavaScript	JavaScript used directly, with no framework or library
Paise	The minor unit of the Indian rupee; ₹1 = 100 paise
IEEE 754	The standard governing floating-point arithmetic in JavaScript
SRS	Software Requirements Specification
FR / NFR	Functional Requirement / Non-Functional Requirement

1.5 References

1. IEEE Std 830-1998, *IEEE Recommended Practice for Software Requirements Specifications*. Institute of Electrical and Electronics Engineers, 1998.
2. WHATWG, *HTML Living Standard*. https://html.spec.whatwg.org
3. MDN Web Docs, *Web Storage API*. https://developer.mozilla.org/en-US/docs/Web/API/Web_Storage_API
4. MDN Web Docs, *JSON*. https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/JSON
5. ECMA International, *ECMAScript 2015 Language Specification (ECMA-262, 6th Edition)*, June 2015.
6. IEEE Std 754-2019, *IEEE Standard for Floating-Point Arithmetic*.

1.6 Overview of the Document

Section	Contents
2	The system in general terms — its context, users, environment and constraints
3	Functional requirements: what the system does, stated individually and testably
4	Non-functional requirements: qualities the system must exhibit
5	Data requirements: what is stored, and how it is structured
6	Interface requirements: what each screen must let the user do
7	The balance and settlement algorithm, in full
8	Business rules and validation
9	Test cases and acceptance criteria
10	Honest statement of what the system cannot do
11	Enhancements possible in a future version
12	Conclusion

Appendices contain the detailed data model, worked examples, the full test catalogue, consolidated validation rules, a glossary, and a traceability matrix.

On the structure. The organisation of this document follows the recommendations of IEEE 830-1998, adapted to the scope of a single-developer frontend project. Sections that standard suggests but which do not apply here — among them multi-user interface requirements and hardware interfaces beyond a browser — are either omitted or reduced to a statement that they do not apply. No claim of strict conformance to the standard is made.

---

2. OVERALL DESCRIPTION

2.1 Product Perspective

SettleUp is a self-contained product. It is not a component of any larger system and depends on no external service.

The application is expected to be organised roughly as follows. The file names are indicative and may change:

File	Responsibility
`index.html`	Page structure
`css/style.css`	All styling
`js/store.js`	Application state; reading and writing localStorage
`js/calc.js`	Share, balance and settlement calculation

File	Responsibility
`js/ui.js`	Rendering and event handling

This is a file organisation, not a formal architecture, and it is recorded here for one reason only: `js/calc.js` contains no reference to the DOM. The settlement calculation is the most error-prone part of the project, and keeping it separate means it can be checked directly in the browser console by supplying balances and inspecting the result, without going through the interface at all. NFR-25 and NFR-26 make this separation a requirement.

The exact file names may change during implementation. The separation of calculation from rendering shall not.

2.2 Product Functions

ID	Function	Summary
PF-01	Member management	Add and display members; remove them where permitted
PF-02	Expense recording	Record an expense with all its attributes
PF-03	Expense splitting	Divide an expense equally, by exact amounts, or by percentage
PF-04	Balance calculation	Compute the net financial position of every member
PF-05	Settlement generation	Produce a reduced list of payments that clears all balances
PF-06	Expense history	List all recorded expenses
PF-07	Search and filtering	Locate expenses by text, payer, or category
PF-08	Category-wise spending	Total spending grouped by category
PF-09	Persistence	Save state to, and restore it from, localStorage
PF-10	Export and import	Write state to a JSON file and read it back

2.3 Intended User

The application has one class of user: a member of a group who takes responsibility for recording the group's shared expenses.

Attribute	Description
Technical background	General familiarity with using websites and mobile apps. No programming or accounting knowledge assumed.
Domain knowledge	Understands informally what it means to split a bill
Pattern of use	Intermittent. Several entries over the course of a trip or shared period, followed by a single settlement at the end.
Device	Likely a mobile phone while entering expenses; possibly a laptop when reviewing the settlement

Because use is intermittent, the interface must be understandable on returning to it after a gap of days. No tutorial or guided onboarding is planned; the screens themselves must carry the necessary explanation.

2.4 Operating Environment

Aspect	Requirement
Platform	Any device with a modern web browser
Browsers	Recent versions of Chrome, Firefox, Safari and Edge, on both desktop and mobile
Required browser features	ECMAScript 2015, the Web Storage API, and the File API (for export/import)
Network	Required only to load the page initially, and only if the page is hosted remotely. Opening the files locally requires no network at all.
Installation	None. The application is opened by loading `index.html`.
Server	None

2.5 Design and Implementation Constraints

ID	Constraint	Origin
C-01	The application shall be built using HTML5, CSS3 and vanilla JavaScript (ECMAScript 2015) only	Academic requirement of the course
C-02	No JavaScript framework or UI library shall be used — specifically not React, Angular, Vue, jQuery, Bootstrap or Tailwind	Academic requirement
C-03	No third-party JavaScript library shall be used. Everything the application does — including any chart, any date handling and any validation — shall be written by the developer	Academic requirement; the objective OBJ-05 is to demonstrate competence without such aids
C-04	The application shall have no server-side component	Project is defined as frontend-only
C-05	The application shall require no build step, bundler, compiler or package installation	It must run by opening a file in a browser
C-06	All persistent data shall be stored in browser localStorage	No database is available
C-07	Monetary calculation shall be performed in a manner that avoids the accumulation of floating-point error — see DR-09	Correctness requirement; JavaScript numbers are IEEE 754 doubles and cannot represent all decimal fractions exactly
C-08	The application shall function correctly when the files are opened directly from the local filesystem	Enables offline use and simplifies evaluation

2.5.1 Rationale for Key Technology Decisions

These are recorded here because they are likely to be questioned during evaluation.

Why vanilla JavaScript rather than a framework. The course requires it, and the objective is to demonstrate direct competence with the DOM, events and state. A framework would hide precisely the mechanisms the project is intended

to exercise. The application's scale — a handful of screens over a small dataset — does not create the complexity that frameworks exist to manage.

Why localStorage rather than a database. The project is frontend-only, so no database server is available. localStorage is the browser's built-in facility for persisting data across sessions. It is adequate here because the data is small, belongs to one user, and never needs to be shared. Its limitations are stated honestly in Section 10.2.

Why no backend. A backend would introduce hosting, deployment and server-side language requirements outside the scope of this course. Every function of SettleUp can be performed on the client, so no backend is needed. The consequences of that choice — no synchronisation, no backup, no sharing — are recorded in Section 10.

2.6 Assumptions and Dependencies

Assumptions

ID	Assumption	Consequence if false
A-01	The user's browser supports ECMAScript 2015 and the Web Storage API	The application will not run
A-02	JavaScript is enabled	The application will not run
A-03	One person operates the application on behalf of the whole group	Concurrent edits are neither detected nor merged
A-04	All amounts within the application are in a single currency	Mixed-currency totals would be meaningless
A-05	The user does not clear browser storage or site data between sessions	Stored data would be lost
A-06	The user opens the application from the same browser and same device each time	Data would not be found
A-07	Group sizes are those typical of shared trips or households	Not a hard limit; the design imposes none, but no requirement is made about behaviour at very large sizes

Dependencies

The application depends only on the browser runtime — specifically on the Web Storage API, the File API and the DOM. It depends on no library, no framework, no server and no external service.

Where localStorage is unavailable — as in some private browsing modes, or when site data is blocked — the application shall remain usable for the current session and shall inform the user that data will not be retained (FR-18).

2.7 Project Scope Boundaries

Requirements in this document fall into three categories. This classification exists so that no feature is silently promised.

Category	Meaning
Planned	Committed for this version. Absence would mean the project is incomplete.
Recommended	Not in the original feature list, but closely related to a planned feature and worth implementing if time permits. Stated separately in Section 3.14 under the prefix **RF**, never within the mandatory FR series.
Future	Explicitly deferred. Documented in Section 11 only, and never described as current functionality.

2.7.1 Planned Features

Member management · Expense recording · Equal split · Exact-amount split · Percentage split · Balance calculation · Settlement generation · Expense history · Search and filtering · Category-wise spending · localStorage persistence · JSON export and import.

2.7.2 Recommended Features

Feature	Why recommended	Related planned feature
Delete an expense	A mistyped expense corrupts every balance until it can be removed. Without this, the only remedy is to clear all data.	Expense recording
Sort expense history	The history is already filterable; sorting is a small addition to the same view	Expense history
Graphical display of category spending	Category totals are planned; presenting them as a simple bar chart is a presentation choice on top of the same data	Category-wise spending

Recommended features are specified in **Section 3.14** only. They carry **RF** identifiers rather than FR, and are worded with *should* rather than *shall*. They do not appear anywhere in Section 3.1 to 3.13, and they are not counted in the acceptance criteria of Section 9.10.

The purpose of this separation is that a reader of Section 3 sees only what is committed. Nothing in the mandatory requirements depends on a recommended feature being present.

2.7.3 Deferred to Future Versions

Editing an existing expense · renaming a member · undo · multiple groups · multi-currency support · data schema migration · any form of automated test framework. These appear in Section 11 only.

---

3. FUNCTIONAL REQUIREMENTS

Requirements in this section state what the system does. Each is written so that it can be checked by observing the running application.

The word **shall** is used throughout to indicate a mandatory requirement. This follows IEEE 830 convention: *shall* denotes an obligation, whereas *should* would denote a recommendation and *may* an option.

Priority is recorded as:

- **High** — the system is not functional without it
- **Medium** — the system works, but an objective is not met
- **Low** — a convenience

3.1 Member Management

FR-01 — Add a Member

Description	The system shall allow the user to add a member to the group by entering a name.
Input	Member name (text)
Processing	Remove leading and trailing whitespace. Verify the result is not empty. Assign the next unused numeric ID. Append to the member list. Persist.
Output	The new member appears in the member list and becomes available for selection as payer and participant.
Validation	Name must not be empty after trimming. Governed by BR-01 and BR-02.
Error and edge cases	Empty or whitespace-only input → rejected with the message "Please enter a name". A name identical to an existing member's name is **accepted**: the two are separate members with separate identifiers, and the interface distinguishes them (UI-06).
Priority	High
Acceptance criteria	AC-01: Adding a valid name causes it to appear in the member list. AC-02: Adding a name identical to an existing member creates a second, separate member, and expenses recorded against one do not affect the other.

FR-02 — Assign Unique Member Identifiers

Description	The system shall assign every member a unique numeric identifier at the time of creation, and shall use that identifier for every internal reference to the member.
Processing	A counter is maintained in application state. Each new member takes the counter's current value; the counter is then incremented. Identifiers are never reused, including after a member is removed.
Output	Every member holds an `id` which is unique and never changes
Priority	High
Acceptance criteria	AC-03: Two members with identical names hold different identifiers, and expenses recorded against one do not affect the other. This case is reachable, since FR-01 permits duplicate names.

The rationale for identifiers is given in Section 5.4.1.

FR-03 — Display Members

Description	The system shall display all members of the group.
Output	A list showing each member's name
Error and edge cases	Where no members exist, the system shall display a message explaining that members must be added before expenses can be recorded, rather than an empty area.
Priority	High
Acceptance criteria	AC-04: Every added member appears in the list. AC-05: With no members, an explanatory message is shown.

FR-04 — Remove a Member

Description	The system shall allow the user to remove a member who is not referenced by any expense.
Input	Selection of a member to remove
Processing	Check whether the member's identifier appears as the payer of any expense, or within the shares of any expense. If it does not, remove the member and persist. If it does, refuse.
Output	The member is removed from the list, or a message explains why removal is not possible.
Validation	Governed by BR-03
Error and edge cases	Attempting to remove a member referenced by an expense → refused, with a message stating that the member appears in recorded expenses.
Priority	Medium
Acceptance criteria	AC-06: A member with no expenses can be removed. AC-07: A member referenced by an expense cannot be removed, and a reason is shown.

Design note. The alternative — deleting the member and every expense involving them — would silently destroy data the user may not expect to lose. Refusing removal is the safer behaviour, and it preserves the integrity constraint DR-11.

3.2 Expense Management

FR-05 — Record an Expense

Description	The system shall allow the user to record an expense against the group.
Input	Description (text); amount (number); payer (a member); category (text); participants (one or more members); split method (equal, exact or percentage); and, for exact and percentage methods, the individual split values
Processing	Validate every field. Compute each participant's share according to the selected split method (Sections 3.3 to 3.5). Assign a unique numeric expense identifier and record a timestamp. Append to the expense list. Persist. Recompute balances.
Output	The expense appears in the history; all balances are updated
Validation	Description not empty. Amount a number greater than zero. A payer selected. At least one participant selected. Split values valid for the chosen method. Governed by BR-05 to BR-09.

Error and edge cases	Amount of zero or negative → rejected. Non-numeric amount → rejected. No participants selected → rejected. Each rejection displays a message identifying the field at fault, and no expense is created.
Priority	High
Acceptance criteria	AC-08: A valid expense appears in the history and changes the balances. AC-09: An invalid expense is refused with a field-specific message and does not appear in the history.

FR-06 — Payer Need Not Be a Participant

Description	The system shall permit an expense whose payer is not among its participants.
Rationale	A member may pay for something they take no share of — for example, paying for a meal they did not eat.
Processing	The payer's balance increases by the full amount. Their share is zero unless they also appear as a participant.
Priority	Medium
Acceptance criteria	AC-10: Where A pays ₹300 split between B and C, A's balance becomes +300, and B and C each become −150.

3.3 Equal Split

FR-07 — Divide an Expense Equally

Description	The system shall divide an expense equally among its participants.
Input	Expense amount; set of participants
Processing	Divide the amount by the number of participants. Where the amount does not divide exactly into whole minor units, distribute the remainder one minor unit at a time to participants in order, so that the shares sum exactly to the expense amount. The full method is specified in Section 7.6.2.
Output	One share per participant; the shares total exactly the expense amount
Validation	At least one participant. Governed by BR-10 and BR-11.
Error and edge cases	An amount not evenly divisible — for example ₹100 among three participants — must not lose or create money through rounding.
Priority	High
Acceptance criteria	AC-11: ₹100 split equally among three participants produces shares of ₹33.34, ₹33.33 and ₹33.33, totalling exactly ₹100.00.

Why this matters. Rounding each share independently to ₹33.33 would produce a total of ₹99.99, losing one paisa from the group. Over many expenses these losses accumulate and the balances no longer sum to zero, which breaks the settlement algorithm's central assumption. Remainder distribution prevents this.

3.4 Exact Amount Split

FR-08 — Divide an Expense by Exact Amounts

Description	The system shall allow the user to specify each participant's share directly.
Input	Expense amount; for each participant, an amount
Processing	Verify each specified amount is a number and is not negative. Verify the specified amounts total exactly the expense amount. Record them as the shares.
Output	Shares exactly as specified
Validation	Sum of shares equals the expense amount. No share is negative. Governed by BR-12 and BR-13.
Error and edge cases	Where the sum does not match, the system shall reject the expense and display both the current total and the difference, so the user can see how much is unaccounted for.
Priority	High
Acceptance criteria	AC-12: Where shares total the expense amount, the expense is accepted. AC-13: Where they do not, it is refused and the shortfall or excess is displayed.

Use case. Four people share a restaurant bill of ₹1,200, but one ordered only a drink. Equal splitting would be unfair; exact amounts allow ₹50, ₹400, ₹400 and ₹350.

3.5 Percentage Split

FR-09 — Divide an Expense by Percentage

Description	The system shall allow the user to specify each participant's share as a percentage of the expense.
Input	Expense amount; for each participant, a percentage
Processing	Verify each percentage is a number and is not negative. Verify the percentages total exactly 100. Compute each share as `amount × percentage ÷ 100`. Apply the remainder distribution of Section 7.6.2 so that the resulting shares total exactly the expense amount.
Output	One share per participant, totalling exactly the expense amount
Validation	Percentages total 100. No percentage is negative. Governed by BR-14 to BR-16.
Error and edge cases	Where percentages do not total 100, the expense shall be rejected and the running total displayed. Where the computed shares do not total the amount after rounding, the remainder is distributed as in FR-07.
Priority	High
Acceptance criteria	AC-14: Percentages totalling 100 produce shares totalling exactly the expense amount. AC-15: Percentages not totalling 100 are refused, with the running total shown.

Use case. Three people share a room costing ₹9,000. One has the double bed and two share the other; a 50/25/25 split is agreed.

3.6 Balance Calculation

FR-10 — Compute Net Balances

Description	The system shall compute the net balance of every member.
Input	The complete list of members and the complete list of expenses
Processing	For each member: total the amounts of all expenses where they are the payer; subtract the total of their shares across all expenses. The method is specified in Section 7.2.
Output	A net balance per member, each classified as creditor (positive), debtor (negative) or settled (zero)
Validation	The sum of all net balances shall be zero — see BR-17
Error and edge cases	Where the sum is not zero, the system shall report a diagnostic message rather than displaying a settlement derived from inconsistent data.
Priority	High
Acceptance criteria	AC-16: Balances match manual calculation for the worked example in Appendix B. AC-17: The sum of all balances is zero for every set of expenses.

FR-11 — Balances Reflect All Recorded Expenses

Description	The system shall ensure that the balances it displays always correspond to the complete set of currently recorded expenses, including after an expense has been added or removed.
Priority	High
Acceptance criteria	AC-18: Balances displayed after adding several expenses equal balances computed from the same expense list loaded fresh.

Design decision (DD-01). Balances will be derived from the expense list rather than stored as independent running totals.

This is recorded separately from the requirement above because it is a means of meeting that requirement, not an observable behaviour in itself. The requirement is what will be tested; this is how it will be achieved.

The reason for the choice: a running total that becomes incorrect — through a bug or a rounding error — cannot repair itself, and every value computed after it inherits the error. Deriving balances from the expense list makes the expenses the single source of truth, so a fault in one calculation does not survive into the next.

3.7 Settlement Generation

FR-12 — Generate a Settlement Plan

Description	The system shall generate a list of payments which, when made, reduces every member's net balance to zero.
Input	The net balance of every member
Processing	Greedy creditor–debtor matching, specified in full in Section 7.
Output	An ordered list of payments, each stating a payer, a payee and an amount, expressed in plain language — for example, "Karan pays Amit ₹2,500.00"
Validation	Governed by BR-19 to BR-22
Error and edge cases	Where all balances are zero, the system shall state that the group is already settled and display no payments. Where only one member exists, the same applies.
Priority	High
Acceptance criteria	AC-19: Applying every payment in the generated list brings all balances to zero. AC-20: The list contains at most one payment fewer than the number of members. AC-21: No payment has an amount of zero.

3.8 Expense History

FR-13 — Display Expense History

Description	The system shall display all recorded expenses.
Output	A list showing, for each expense: description, amount, payer, category and date. Selecting an expense reveals the individual shares.
Processing	Expenses are shown in reverse chronological order by default, most recent first.
Error and edge cases	Where no expenses exist, an explanatory message is displayed rather than an empty area.
Priority	High
Acceptance criteria	AC-22: Every recorded expense appears in the history with its correct details.

3.9 Search and Filtering

FR-14 — Search and Filter Expenses

Description	The system shall allow the user to locate expenses by searching descriptions and by filtering on payer and category.
Input	A search term; a selected payer; a selected category
Processing	Match the search term against expense descriptions as a substring, ignoring case. Apply payer and category filters. Where more than one is active, apply them together.
Output	The history list, showing only matching expenses
Error and edge cases	Where no expense matches, the system shall display a message stating that nothing matched, and shall offer a means of clearing the filters.

Priority	Medium
Acceptance criteria	AC-23: Searching a term shows only expenses whose descriptions contain it. AC-24: Combining a search term with a category filter shows only expenses satisfying both.

3.10 Category-wise Spending

FR-15 — Display Spending by Category

Description	The system shall display the total spent in each category, and the overall group total.
Input	The complete expense list
Processing	Group expenses by category; total the amounts within each group; total across all groups.
Output	A list of categories with their totals, and the group total
Error and edge cases	Where no expenses exist, all totals are zero and this is stated rather than left blank.
Priority	Medium
Acceptance criteria	AC-25: Category totals sum to the group total. AC-26: Adding an expense in a category increases that category's total by the expense amount.

3.11 localStorage Persistence

FR-16 — Save Application State

Description	The system shall write the complete application state to localStorage whenever the state changes.
Processing	Serialise the state to a JSON string; write it under a fixed key.
Error and edge cases	Where the write fails — for example because storage is full or unavailable — the failure shall not interrupt the user's work. The in-memory state remains correct for the session, and the user is informed.
Priority	High
Acceptance criteria	AC-27: After adding a member and an expense, the stored value contains both.

FR-17 — Restore Application State

Description	The system shall restore the application state from localStorage when the page loads.
Processing	Read the stored string; parse it; verify it has the expected structure; adopt it as the application state.
Error and edge cases	Where no stored value exists, the application starts empty. Where the stored value cannot be parsed, or does not have the expected structure, the application shall start empty rather than fail — see BR-23.
Priority	High
Acceptance criteria	AC-28: After a page reload, all members and expenses are present. AC-29: With deliberately corrupted stored data, the application loads and is usable, starting empty.

FR-18 — Behaviour Where Storage Is Unavailable

Description	Where localStorage cannot be accessed, the system shall continue to operate for the current session and shall inform the user that data will not be retained.
Rationale	Some private browsing modes and some browser configurations block storage. Failing entirely would be a worse outcome than operating without persistence.
Priority	Medium
Acceptance criteria	AC-30: With storage blocked, the application loads, accepts input, and displays a notice about non-retention.

3.12 JSON Export

FR-19 — Export Data as JSON

Description	The system shall allow the user to download the complete application state as a JSON file.
Input	A user action requesting export
Processing	Serialise the state to formatted JSON; create a downloadable file; trigger the download.
Output	A `.json` file saved to the user's device
Error and edge cases	Where there is no data, the system shall either export an empty structure or state that there is nothing to export.
Priority	Medium
Acceptance criteria	AC-31: The exported file is valid JSON and contains every member and expense.

Purpose. Export is the only means by which data can leave one browser and reach another. It serves as backup, as transfer between devices, and as evidence during evaluation.

3.13 JSON Import

FR-20 — Import Data from JSON

Description	The system shall allow the user to load a previously exported JSON file, replacing the current application state.
Input	A JSON file chosen by the user

Processing	Read the file; parse it; validate the structure against the rules of Section 8.8; on success, replace the state and persist; on failure, leave the existing state untouched.
Output	The imported data appears throughout the application, or an error message explains why the file was rejected
Validation	Governed by BR-24 to BR-26
Error and edge cases	File is not valid JSON → rejected. Required fields missing → rejected. An expense references a member identifier that does not exist in the file → rejected. In every case, the existing data remains unchanged.
Priority	Medium
Acceptance criteria	AC-32: A file previously produced by export imports successfully and reproduces the same state. AC-33: A malformed or inconsistent file is refused with a message, and existing data is preserved.

FR-21 — Confirm Before Replacing Data

Description	Before an import replaces existing data, the system shall require the user to confirm, stating clearly that current data will be replaced.
Rationale	Import is destructive and, without an undo facility, irreversible.
Priority	Medium
Acceptance criteria	AC-34: Import does not proceed until the user confirms.

3.14 Recommended Features

The requirements in this subsection are **not mandatory**. They are not counted in the acceptance criteria of Section 9.10, and the project is complete without them. They are recorded because each is closely related to a planned feature and would be worth adding if time permits.

They use the prefix **RF** rather than FR so that no recommended item can be mistaken for a committed requirement, and they use **should** rather than **shall**.

RF-01 — Delete an Expense [Recommended]

Description	The system should allow the user to delete a recorded expense.
Processing	Remove the expense from the list, persist, and recompute all balances from the remaining expenses.
Rationale	Without this, a single mistyped amount cannot be corrected except by clearing all data.
Priority	Medium — Recommended, not required for acceptance
Acceptance criteria	Where implemented: deleting an expense restores balances to the values they held before that expense was recorded.

RF-02 — Sort Expense History [Recommended]

Description	The system should allow the history to be sorted by date or by amount, ascending or descending.
Priority	Low — Recommended

RF-03 — Graphical Category Breakdown [Recommended]

Description	The system should present the category breakdown graphically, in addition to the numeric totals.
Note	If implemented, this shall be drawn using HTML and CSS, or SVG generated by the application. No charting library shall be used — see C-03.
Priority	Low — Recommended

---

4. NON-FUNCTIONAL REQUIREMENTS

Functional requirements state what the system does. Non-functional requirements state qualities it must possess while doing it.

The requirements below are deliberately stated in terms that can be checked by observation on the target browsers. Numeric thresholds have been avoided where no measurement has been performed to justify them, since an unjustified number is not a requirement but a guess.

4.1 Usability

ID	Requirement	How verified
NFR-01	Every form field shall have a visible label describing what it expects	Inspection
NFR-02	Validation errors shall be displayed in text, adjacent to the field concerned, and shall state both what is wrong and what the user should do	Inspection of each error path in Appendix C
NFR-03	Error information shall not be conveyed by colour alone	Inspection
NFR-04	Every list view shall have a defined empty state explaining what the user should do next	Inspection of each list with no data
NFR-05	Actions that destroy data — removing a member, importing over existing data — shall require explicit confirmation	Inspection
NFR-06	All monetary values shall be displayed with exactly two decimal places	Inspection
NFR-07	A user shall be able to record an expense without external instruction, using only what is visible on screen	Observation of a first-time user

4.2 Performance

ID	Requirement	How verified
NFR-08	The application shall respond to user actions without perceptible delay for group sizes and expense counts typical of its intended use	Manual observation during testing on both a desktop and a mobile browser
NFR-09	Adding an expense shall update the balances and settlement without a visible pause, so that no loading indicator is needed	Manual observation on one desktop and one mobile browser

Note on the absence of numeric targets. No specific response-time figure is stated because none has been measured, and a figure invented for the document would not be a requirement that could meaningfully be met or missed. The computational work involved — a small number of arithmetic operations per expense, and a sort over the member list — is trivial at the scale this application operates at. If measurement during development shows otherwise, a numeric requirement will be added in a subsequent revision.

4.3 Reliability

ID	Requirement	How verified
NFR-10	A failure to write to localStorage shall not cause loss of data held in the current session	TC-30
NFR-11	Unreadable or malformed stored data shall not prevent the application from starting	TC-29
NFR-12	An invalid import shall leave the existing data unchanged	TC-33
NFR-13	No user action shall leave the application in a state where the displayed balances do not follow from the recorded expenses	Inspection following each test in Appendix C

4.4 Data Integrity

ID	Requirement	How verified
NFR-14	The shares of an expense shall total exactly that expense's amount	TC-11, TC-13
NFR-15	The net balances of all members shall total zero	TC-16
NFR-16	Every member identifier referenced by an expense shall correspond to an existing member	TC-06, TC-33
NFR-17	Monetary arithmetic shall be carried out so that repeated operations do not accumulate floating-point error — see DR-09	TC-11, TC-14

4.5 Browser Compatibility

ID	Requirement	How verified
NFR-18	The application shall function correctly in recent versions of Chrome, Firefox and Edge	Manual testing in each
NFR-19	The application shall function in a mobile browser on Android	Manual testing on one physical Android device
NFR-20	The application shall be usable at viewport widths from approximately that of a small phone to that of a desktop monitor, without horizontal scrolling	Browser device emulation in developer tools

On Safari and iOS. Neither is listed above, and neither is claimed. The development machine runs Windows, on which Safari is unavailable, and no Apple device is available for testing. Nothing in the application is expected to be browser-specific, so it may well work there — but an untested claim would be a claim this project cannot support, and none is made.

4.6 Privacy

ID	Requirement	How verified
NFR-21	The application shall make no network request after the initial page load	Browser developer tools, network panel
NFR-22	The application shall load no third-party script	Inspection of the HTML source
NFR-23	All user data shall remain on the user's own device unless the user explicitly exports it	Follows from NFR-21 and NFR-22
NFR-24	User-supplied text shall be inserted into the page as text, never as markup	Code inspection; TC-05

A necessary clarification. These requirements mean that data is not transmitted anywhere. They do **not** mean the data is secure. Data in localStorage is unencrypted and readable by anyone with access to the browser profile, including through the browser's own developer tools. This application provides privacy in the sense of *non-transmission*, not security in the sense of *protection*. Section 10.4 states this in full.

4.7 Maintainability

ID	Requirement	How verified
NFR-25	Calculation logic shall contain no reference to the DOM	Code inspection
NFR-26	Rendering code shall contain no monetary calculation	Code inspection
NFR-27	Calculation, state and persistence, and rendering shall each reside in a separate JavaScript file	Code inspection
NFR-28	Non-obvious logic shall carry a comment explaining why it is written that way, rather than restating what the code does	Code inspection

4.8 Basic Accessibility

The requirements here are limited to what can realistically be achieved and verified in a project of this scope. No claim of conformance to any accessibility standard is made.

ID	Requirement	How verified
NFR-29	Every form control shall be associated with a `<label>` element	Code inspection
NFR-30	All functionality shall be reachable and operable using the keyboard alone	Manual testing without a mouse
NFR-31	The element with keyboard focus shall be visibly distinguishable	Manual testing
NFR-32	Text shall be presented in dark tones on light backgrounds, or the reverse, so that it is comfortably readable. Colour pairs are reviewed during development using the contrast information in browser developer tools	Development-time review; not a separate test deliverable
NFR-33	Semantic HTML elements shall be used according to their meaning — headings for headings, lists for lists, buttons for actions	Code inspection

---

5. DATA REQUIREMENTS

5.1 Member Data

ID	Requirement
DR-01	A member shall consist of a numeric identifier and a name
DR-02	The identifier shall be unique among members, assigned at creation, and never changed
DR-03	The name shall be a non-empty string after trimming. Names are **not** required to be unique — two members may hold the same name and are distinguished by their identifiers

```
Member
├── id    : number    — unique, assigned at creation, immutable
└── name  : string    — non-empty; need not be unique
```

5.2 Expense Data

ID	Requirement
DR-04	An expense shall consist of an identifier, a description, an amount, a payer, a category, a split method, a set of shares, and a timestamp
DR-05	The identifier shall be unique among expenses and shall never change
DR-06	The payer shall be recorded as a member identifier
DR-07	The amount shall be stored as a positive integer number of paise — see DR-09
DR-08	The timestamp shall record when the expense was entered, in ISO 8601 format

```
Expense
├── id          : number    — unique, immutable
├── description : string    — non-empty
├── amount      : number    — integer, paise, greater than zero
├── paidBy      : number    — a Member.id
├── category    : string    — non-empty
├── splitMethod : string    — "equal" | "exact" | "percentage"
├── shares      : Share[]   — one entry per participant
└── timestamp   : string    — ISO 8601
```

DR-09 — Monetary Precision

JavaScript represents all numbers as IEEE 754 double-precision floating point. Decimal fractions such as 0.1 cannot be represented exactly, with the consequence that `0.1 + 0.2` evaluates to `0.30000000000000004`. Accumulated across many additions, this drift can cause the sum of balances to differ from zero, which would violate NFR-15 and invalidate the settlement algorithm's central assumption.

The strategy adopted: all internal monetary arithmetic shall be performed on integers representing paise. An amount entered as ₹45.50 is converted to 4550 on entry. Shares, balances and settlement amounts are computed entirely in paise. Conversion back to rupees occurs only at the point of display.

Why this works. Integer arithmetic in JavaScript is exact for all values within the safe integer range, which is far larger than any total this application will handle. No rounding error can arise, because no fractional values are ever held.

This extends to storage and export. Amounts are held as paise in localStorage and in the exported JSON as well, not only in memory. Storing rupees and converting on load would reintroduce the very conversion — a decimal string multiplied by 100 — that working in paise exists to avoid. The consequence is that an exported file shows 600000 where the user entered ₹6,000; this is documented in Section 5.7 so that anyone reading an export knows how to interpret it.

The alternative considered. Rounding every intermediate result to two decimal places would also work, but requires care at every arithmetic step and is easy to get wrong in one place. Working in paise makes the property automatic rather than something that must be remembered.

5.3 Share Data

ID	Requirement
DR-10	A share shall consist of a member identifier and an amount
DR-11	Every member identifier in a share shall correspond to an existing member
DR-12	The shares of an expense shall total exactly that expense's amount

```
Share
├── memberId : number    — a Member.id
└── amount   : number    — integer, paise, the participant's portion
```

5.4 Data Relationships

```
Member  1 ──────< paidBy      Expense
Member  1 ──────< memberId    Share
Expense 1 ──────< shares      Share
```

An expense has exactly one payer and one or more shares. A member may be the payer of many expenses and may appear in the shares of many expenses.

5.4.1 Why Member Identifiers Rather Than Names

This is the most consequential data decision in the project, and it is one an evaluator is likely to probe.

The decision. Expenses reference members by numeric identifier. A member's name appears in exactly one place — the member record itself.

Reason 1 — names are not unique. Two people in a group may share a name, and FR-01 deliberately permits this rather than forcing the user to invent a distinguishing spelling. If an expense recorded `paidBy: "Rahul"` and the group contains two people called Rahul, the system could not determine which one paid. Numeric identifiers are unique by construction, so this ambiguity cannot arise. This is the reason the design decision is real rather than theoretical: the situation it guards against is one the application actually allows.

Reason 2 — names change. If a name were used as the reference and the user later corrected a spelling, every expense referring to the old spelling would become an orphan. With identifiers, the name can change freely and every reference remains valid. This is what makes the future *rename member* enhancement (Section 11) straightforward rather than a redesign.

Reason 3 — comparison is cheap and unambiguous. Comparing two numbers requires no decision about case, whitespace or accented characters. Comparing two strings requires all of those decisions, and each is a place a bug can hide.

Worked illustration.

```
Members:
  { id: 1, name: "Rahul" }
  { id: 2, name: "Rahul" }      ← same name, different person

Expense:
  { id: 1, description: "Petrol", amount: 800, paidBy: 1, ... }
                                                 └── unambiguous
```

Had the expense stored `paidBy: "Rahul"`, no rule could recover which of the two paid.

5.5 Validation Rules

The validation rules governing data are consolidated in Section 8 and listed in full in Appendix D.

5.6 localStorage Representation

ID	Requirement
DR-13	The complete application state shall be stored as a single JSON string under a single fixed key
DR-14	The stored value shall be written whenever the state changes and read once at page load
DR-15	The stored value shall not be relied upon as valid; it shall be checked before use — see BR-10

localStorage stores only strings. The application state is therefore converted to a JSON string on save and parsed back on load. A single key is used rather than several, so that a save either succeeds entirely or fails entirely, and the stored data can never represent a half-updated state.

Structure of the stored value:

```
{
  "members": [
    { "id": 1, "name": "Amit" },
    { "id": 2, "name": "Riya" }
  ],
  "expenses": [
    {
      "id": 1,
      "description": "Hotel",
      "amount": 600000,
      "paidBy": 1,
      "category": "Accommodation",
      "splitMethod": "equal",
      "shares": [
        { "memberId": 1, "amount": 200000 },
        { "memberId": 2, "amount": 200000 },
        { "memberId": 3, "amount": 200000 }
      ],
      "timestamp": "2026-08-14T10:30:00.000Z"
    }
  ],
  "nextMemberId": 3,
  "nextExpenseId": 2
}
```

The two counters are stored alongside the data so that identifiers remain unique across sessions. Without them, a reloaded application would begin assigning identifiers from 1 again and would collide with existing records.

5.7 JSON Import and Export Structure

ID	Requirement
DR-16	The exported file shall use the same structure as the stored value shown in Section 5.6
DR-17	Imported data shall be validated against that structure before it is adopted

Using one structure for both storage and transfer means there is a single format to define, document and validate, rather than two that could drift apart.

Reading an exported file. All amounts in the file are integer paise, as specified in DR-09. A value of `600000` is ₹6,000.00. Conversion to rupees happens only at the point of display within the application.

---

6. SYSTEM AND USER INTERFACE REQUIREMENTS

This section states what each part of the interface must allow the user to do. It deliberately does not specify visual design — layout, colour and typography are not yet decided and will be determined during implementation.

6.1 Overall Interface

ID	Requirement
UI-01	The application shall present its functions as distinct sections or views, navigable without a page reload
UI-02	The current view shall be identifiable at all times
UI-03	The layout shall adapt to the viewport width, from small phone to desktop, without horizontal scrolling
UI-04	Monetary values shall be displayed with two decimal places and a currency symbol

6.2 Member Interface

ID	Requirement
UI-05	Shall provide a field for entering a name and a control for adding it
UI-06	Shall display all members. Where two or more members hold the same name, the interface shall distinguish them — for example by appending a sequence number
UI-07	Shall provide a means of removing a member, subject to FR-04
UI-08	Shall display validation errors adjacent to the entry field
UI-09	Shall display an explanatory message when no members exist

6.3 Expense Entry Interface

ID	Requirement
UI-10	Shall provide fields for description, amount and category
UI-11	Shall provide a means of selecting the payer from among the members, distinguishing members who share a name as in UI-06
UI-12	Shall provide a means of selecting one or more participants
UI-13	Shall provide a means of choosing the split method
UI-14	Where the exact or percentage method is chosen, shall reveal a value field for each selected participant
UI-15	For the exact and percentage methods, shall display a running total of the entered values, so that the user can see whether they are complete before submitting
UI-16	Shall display validation errors adjacent to the field concerned
UI-17	Shall be unavailable, with an explanation, when fewer than one member exists

6.4 Balance Interface

ID	Requirement
UI-18	Shall display every member's net balance
UI-19	Shall distinguish creditors, debtors and settled members by a means other than colour alone
UI-20	Shall express each balance in plain language — for example, "Amit is owed ₹3,500.00" — rather than as a bare signed number

6.5 Settlement Interface

ID	Requirement
UI-21	Shall display the generated payment list
UI-22	Shall express each payment in plain language: payer, payee and amount
UI-23	Shall state the number of payments required
UI-24	Shall state clearly, when all balances are zero, that the group is settled
UI-25	Shall make clear that the application records the required payments but does not perform them

6.6 Expense History Interface

ID	Requirement
UI-26	Shall list all expenses with description, amount, payer, category and date
UI-27	Shall provide a means of viewing the individual shares of a selected expense
UI-28	Shall display an explanatory message when no expenses exist
UI-29	Shall provide search and filter controls, and a means of clearing them

ID	Requirement
UI-30	Shall state clearly when filters are active and no expense matches

6.7 Spending Analysis Interface

ID	Requirement
UI-31	Shall display each category with its total
UI-32	Shall display the overall group total
UI-33	Shall display each member's total contribution

6.8 Import and Export Interface

ID	Requirement
UI-34	Shall provide a control that downloads the current data as a JSON file
UI-35	Shall provide a control that selects a JSON file for import
UI-36	Shall require confirmation before an import replaces existing data, stating what will be lost
UI-37	Shall report the outcome of an import, and on failure shall state the reason

6.9 Responsive Design Considerations

ID	Requirement
UI-38	Interactive controls shall be large enough to operate with a finger on a touch screen
UI-39	Content shall reflow rather than scale down on narrow viewports
UI-40	No horizontal scrolling shall be required at any supported width
UI-41	On narrow viewports, tabular data shall be presented as stacked rows rather than as a side-scrolling table

---

7. BALANCE CALCULATION AND DEBT MINIMIZATION ALGORITHM

This is the principal technical section of the document. The features described elsewhere are, in the main, the recording and display of data. The material in this section is what distinguishes SettleUp from a simple expense list.

7.1 Problem Definition

A group of *n* members has recorded a set of expenses. Each expense was paid by one member and divided among some subset of members.

Two members can therefore be in an unbalanced position: one has paid more than their share and is owed money; another has paid less and owes money. The task is to produce a list of payments that returns every member to a balanced position.

The naive solution. For every expense, each participant owes the payer their share. Recording each of these as a separate obligation, and then having each debtor pay each creditor directly, produces one payment for every pair of members who owe one another anything. A group of n members has **n(n−1)/2** distinct pairs, so that is the upper bound on how many separate payments such an approach can involve. The number an actual group reaches depends on its expenses; for a group of six the bound is fifteen transfers.

Why this is more than an inconvenience. Each transfer is a real action a real person must take. Fifteen transfers among six friends means fifteen opportunities to forget, to mistype an amount, or to dispute what was sent. Reducing the count reduces all of these.

The observation the solution rests on. Only a member's *net* position matters. If Riya owes ₹500 and is owed ₹500, she need neither pay nor be paid — money would arrive and leave again with no effect. Working from net balances rather than individual debts removes all such pass-through movements automatically.

7.2 Net Balance Calculation

For each member *m*:

```
netBalance(m) = totalPaid(m) − totalShare(m)

where
  totalPaid(m)  = Σ  amount(e)        for every expense e where payer(e) = m
  totalShare(m) = Σ  share(e, m)      for every expense e in which m participates
```

Interpretation:

Net balance	Meaning	Classification
Positive	Paid more than their share; money is owed to them	Creditor
Negative	Paid less than their share; they owe money	Debtor
Zero	Paid exactly their share	Settled

A property worth noting. The sum of all net balances is always zero. Every rupee that appears as a payment by one member appears as a share of one or more members, so the total paid across the group equals the total shared across the group, and the difference is zero.

This is not merely an observation; it is used as a check. Since the property must hold, its failure indicates a fault — a rounding error, a mis-recorded share, or a bug. The application asserts it after every computation (BR-17).

7.3 Creditor and Debtor Identification

Members are partitioned by the sign of their net balance:

```
creditors = { m : netBalance(m) > 0 }
debtors   = { m : netBalance(m) < 0 }
settled   = { m : netBalance(m) = 0 }
```

Settled members take no part in the settlement. Since the balances sum to zero, the creditor set is empty exactly when the debtor set is empty.

7.4 Greedy Settlement Approach

The algorithm proceeds by repeatedly matching the largest creditor with the largest debtor and settling as much as possible between them.

Why "greedy". A greedy algorithm decides using only the situation in front of it at each step, and never revisits that decision. The strategy adopted for this project is to settle the largest amount possible at each step, which is achieved by pairing the largest outstanding creditor with the largest outstanding debtor. This is the strategy chosen here; it is not claimed to be the mathematically best available choice. Section 7.11 states what it does and does not guarantee.

Why this strategy terminates quickly. Settling the maximum possible amount between a pair guarantees that at least one of the two reaches zero and leaves the calculation. Since each step removes at least one member from consideration, the process terminates quickly and produces few payments.

What greedy does not guarantee. It does not guarantee the smallest possible number of payments in every case. Section 7.11 addresses this honestly.

7.5 Step-by-Step Algorithm

1. Compute the net balance of every member.
2. Place members with a positive balance into a creditor list; sort by balance, largest first.
3. Place members with a negative balance into a debtor list; sort by magnitude of balance, largest first.
4. While both lists are non-empty:
5. Take the first creditor *c* and the first debtor *d*.
6. Let *amount* be the smaller of *c*'s balance and the magnitude of *d*'s balance.
7. Record a payment: *d* pays *c* the sum *amount*.
8. Reduce *c*'s balance by *amount*; increase *d*'s balance by *amount*.
9. Remove *c* from the creditor list if its balance is now zero; remove *d* from the debtor list if its balance is now zero.
10. Return the recorded payments.

7.6 Pseudocode

7.6.1 Settlement

```
ALGORITHM  GenerateSettlement
INPUT      balance[1..n]     net balance per member, in paise, summing to 0
OUTPUT     payments          list of (from, to, amount)

 1   creditors ← members with balance > 0, sorted descending by balance
 2   debtors   ← members with balance < 0, sorted descending by |balance|
 3   payments  ← empty list
 4
 5   i ← 0                        index into creditors
 6   j ← 0                        index into debtors
 7
 8   while i < length(creditors) and j < length(debtors) do
 9        c ← creditors[i]
10        d ← debtors[j]
11
12        amount ← min( balance[c], −balance[d] )
13
14        append ( d, c, amount ) to payments
15
16        balance[c] ← balance[c] − amount
17        balance[d] ← balance[d] + amount
18
19        if balance[c] = 0 then i ← i + 1
20        if balance[d] = 0 then j ← j + 1
21   end while
22
23   return payments
```

At line 12 the amount transferred is the smaller of the two magnitudes, so at least one of the two balances becomes zero at lines 16–17, and at least one index advances at lines 19–20. Note that the sorted order remains valid as the loop proceeds: only the two members at the current indices are ever modified, and each is either exhausted or reduced while remaining at the front of its list.

7.6.2 Remainder Distribution for Equal Splits

```
ALGORITHM  DistributeEqually
INPUT      amount            total in paise
           participants      list of member identifiers
OUTPUT     shares            list of (memberId, amount in paise)

 1   n         ← length(participants)
 2   base      ← floor( amount / n )
 3   remainder ← amount − ( base × n )
 4   shares    ← empty list
 5
 6   for k ← 0 to n − 1 do
 7        share ← base
 8        if k < remainder then
 9             share ← share + 1
10        end if
11        append ( participants[k], share ) to shares
12   end for
13
14   return shares
```

Because `remainder` is always less than `n`, the loop adds exactly one extra paisa to exactly `remainder` participants. The shares therefore total `base × n + remainder`, which equals `amount` exactly.

7.7 Worked Example

Group: Amit, Riya, Karan

Expenses:

#	Description	Amount	Paid by	Split
1	Hotel	₹6,000	Amit	Equal, all three
2	Dinner	₹1,500	Riya	Equal, all three

Step 1 — shares

Member	Hotel	Dinner	Total share
Amit	₹2,000	₹500	₹2,500
Riya	₹2,000	₹500	₹2,500
Karan	₹2,000	₹500	₹2,500

Step 2 — net balances

Member	Total paid	Total share	Net balance	Classification
Amit	₹6,000	₹2,500	**+₹3,500**	Creditor
Riya	₹1,500	₹2,500	**−₹1,000**	Debtor
Karan	₹0	₹2,500	**−₹2,500**	Debtor

Check: +3,500 − 1,000 − 2,500 = 0 ✓ (BR-17 satisfied)

Step 3 — settlement

Iteration	Creditors (sorted)	Debtors (sorted)	Amount	Payment recorded
1	Amit +3,500	Karan −2,500, Riya −1,000	min(3500, 2500) = ₹2,500	Karan pays Amit ₹2,500
2	Amit +1,000	Riya −1,000	min(1000, 1000) = ₹1,000	Riya pays Amit ₹1,000

Both lists are now empty; the algorithm terminates.

Result: 2 payments for 3 members. The bound of n−1 = 2 is met.

Further examples, including a case where the naive approach performs markedly worse, appear in Appendix B.

7.8 Edge Cases

Case	Required behaviour
No members	No balances; no settlement; explanatory message
One member	That member's balance is zero regardless of expenses recorded, since they are the payer and the sole participant; no payments
All balances zero	Empty payment list; the group is reported as settled
One member paid for everything, split equally	n−1 payments, all directed to that member
Every member paid an equal amount	All balances zero; no payments
Chained obligations (A owes B, B owes C, equal amounts)	Reduced to a single payment from A to C; B does not appear
Payer is not a participant	The payer's balance increases by the full amount; their share is zero
A member takes part in no expense	Their balance is zero; they do not appear in the settlement
Amount not evenly divisible	Remainder distributed per Section 7.6.2; shares total exactly

7.9 Transaction Count Analysis

This subsection must be read carefully, because it is the point at which a common error arises.

What is being counted. The number of *payments a person must make* — real transfers of money between people. This is not a measure of how long the program takes to run.

The naive figure. Settling every debtor against every creditor separately can produce up to **n(n−1)/2** payments, since that is the number of distinct pairs among n members.

The figure achieved. The algorithm produces at most **n−1** payments.

Why. Each iteration of the loop records exactly one payment and advances at least one index. There are n members in total across both lists, and the loop cannot continue once either list is exhausted. The last remaining member's balance is necessarily zero, since the balances sum to zero throughout. The loop therefore runs at most n−1 times.

Comparison:

Members	Naive worst case: n(n−1)/2	This algorithm: at most n−1
3	3	2
4	6	3
5	10	4
6	15	5
8	28	7
10	45	9

An explicit warning against a misstatement. It would be incorrect to describe this as "reducing complexity from $O(n^2)$ to O(n)". Those terms describe how the *running time* of an algorithm grows with input size. What is reduced here is the *number of payments in the output*. The two are separate properties, and the running time of the settlement algorithm is given in the next subsection.

7.10 Time and Space Complexity

Balance calculation. Each expense is examined once, and within it each share once. For *n* members and *e* expenses, where each expense has at most *n* shares, this is **O(n × e)** in the worst case and **O(e)** where the number of participants per expense is small and does not grow with the group.

Settlement generation.

Step	Cost
Partition members into creditors and debtors	O(n)
Sort each list	O(n log n)
Main loop — at most n−1 iterations, constant work each	O(n)
Total	**O(n log n)**

The sort dominates.

Space. O(n) for the two lists and the payment list. The payment list holds at most n−1 entries.

A note on the alternative formulation. If, instead of sorting once, the algorithm searched for the largest creditor and largest debtor afresh in each iteration, each search would cost O(n) and the total would be $O(n^2)$. Sorting once and advancing through the lists avoids this. Both are correct; the sorted version is preferred.

7.11 Limitations of the Greedy Approach

The n−1 bound is a **guarantee of a maximum**, not a claim of minimality. This distinction matters, and overstating it would be a defect in this document.

Where greedy is not optimal. Consider four members with balances +100, +100, −100, −100.

The greedy algorithm pairs the first creditor with the first debtor, settling ₹100 and removing both; then the second pair, settling ₹100. Two payments — and here that happens to be optimal.

But consider balances +30, +20, −50, and separately +40, −40 within the same group of five. The optimal settlement recognises two independent zero-sum subgroups and produces three payments. The greedy algorithm, working from a single sorted order, may pair members across those subgroups and produce four. Both settle the group correctly; one uses one payment more.

Why the optimum is not computed. Finding the settlement with the genuinely minimum number of payments requires identifying the largest possible collection of disjoint subsets of members whose balances each sum to zero. This is a partitioning problem, and it is NP-hard: no algorithm is known that solves it efficiently as the number of members grows.

Why greedy is nonetheless the right choice here. The n−1 bound is already a substantial improvement on the n(n−1)/2 pairs a naive approach can involve, and computing a genuinely optimal settlement is intractable for the reason given above.

Statement of the claim. The greedy approach may use more payments than a globally optimal settlement in some balance configurations. The project therefore claims correctness and the n−1 upper bound, not global optimality.

---

8. BUSINESS RULES AND VALIDATION

A business rule is a constraint that holds independently of the software — a fact about the problem domain that the system must respect. A functional requirement says what the system *does*; a business rule says what must always *remain true*.

8.1 Member Rules

ID	Rule	Enforced at
BR-01	A member's name shall not be empty after trimming	FR-01
BR-02	Two members may share a name. Members are distinguished by identifier, never by name	FR-01, FR-02
BR-03	A member referenced by any expense shall not be removed	FR-04

ID	Rule	Enforced at
BR-04	A member identifier, once assigned, shall never change and shall never be reused	FR-02

8.2 Expense Rules

ID	Rule	Enforced at
BR-05	An expense amount shall be greater than zero	FR-05
BR-06	An expense shall have exactly one payer	FR-05
BR-07	An expense shall have at least one participant	FR-05
BR-08	Every member identifier referenced by an expense shall correspond to an existing member	FR-05, FR-20
BR-09	An expense description shall not be empty	FR-05

8.3 Equal Split Rules

ID	Rule	Enforced at
BR-10	Shares shall total exactly the expense amount, with any remainder distributed rather than discarded	FR-07
BR-11	No share shall differ from another by more than one minor unit	FR-07

8.4 Exact Split Rules

ID	Rule	Enforced at
BR-12	The specified shares shall total exactly the expense amount	FR-08
BR-13	No specified share shall be negative	FR-08

8.5 Percentage Split Rules

ID	Rule	Enforced at
BR-14	The specified percentages shall total exactly 100	FR-09
BR-15	No specified percentage shall be negative	FR-09
BR-16	Computed shares shall total exactly the expense amount after remainder distribution	FR-09

8.6 Balance Rules

ID	Rule	Enforced at
BR-17	The net balances of all members shall total zero	FR-10
BR-18	Displayed balances shall at all times correspond to the complete set of recorded expenses	FR-11

8.7 Settlement Rules

ID	Rule	Enforced at
BR-19	Applying every payment in a settlement shall bring all balances to zero	FR-12
BR-20	A settlement shall contain no payment of zero	FR-12
BR-21	A settlement shall contain at most n−1 payments for n members	FR-12
BR-22	No payment shall have the same member as both payer and payee	FR-12

8.8 Import and Export Validation Rules

ID	Rule	Enforced at
BR-23	Stored or imported data shall be validated before use; unusable data shall cause the application to start empty rather than to fail	FR-17
BR-24	Imported data shall be rejected unless it parses as JSON and contains member and expense collections in the expected form	FR-20
BR-25	Imported data shall be rejected if any expense references a member identifier absent from the same file	FR-20
BR-26	A rejected import shall leave existing data unchanged	FR-20
BR-27	Import shall not proceed without explicit user confirmation	FR-21

---

9. TESTING AND ACCEPTANCE CRITERIA

9.1 Testing Strategy

Testing for this project is **manual and browser-based**. No automated testing framework is used, and none is planned; introducing one would fall outside the scope of the course and would conflict with constraint C-03.

Each test case in Appendix C specifies a precondition, an action, and an expected result. Tests are executed by performing the action in the running application and comparing the outcome against the expectation. Results are recorded against the test identifier.

Two supplementary techniques are used where the interface alone cannot show what is required:

- **Browser developer console.** Used to inspect computed balances and settlement output directly, and to confirm that internal values are correct rather than merely that the display appears plausible.

- **Browser storage inspector.** Used to examine the stored JSON, and to corrupt it deliberately when testing the recovery behaviour of FR-17.

9.2 Functional Test Cases

Covering member and expense management. See TC-01 to TC-09 in Appendix C.

9.3 Validation Test Cases

Covering rejection of invalid input across all forms. See TC-04, TC-05, TC-08, TC-09.

9.4 Split Calculation Test Cases

Covering all three split methods, including non-divisible amounts. See TC-10 to TC-15.

9.5 Balance Test Cases

Covering net balance computation and the zero-sum property. See TC-16 to TC-19.

9.6 Settlement Test Cases

Covering payment generation, the n−1 bound, and the chained-debt reduction. See TC-20 to TC-26.

9.7 localStorage Test Cases

Covering save, restore, corruption recovery and unavailable storage. See TC-27 to TC-30.

9.8 JSON Import and Export Test Cases

Covering round-trip fidelity and rejection of invalid files. See TC-31 to TC-35.

9.9 Edge Cases

Covering empty group, single member, and other boundary conditions. See TC-36 to TC-40.

9.10 Acceptance Criteria

The project shall be considered complete when all of the following hold.

ID	Criterion
AC-A	All planned features listed in Section 2.7.1 are implemented and function as specified
AC-B	Every test case in Appendix C passes
AC-C	Balances sum to zero for every combination of expenses tested
AC-D	Shares total exactly the expense amount for every split method, including non-divisible amounts
AC-E	Settlement produces at most n−1 payments and, when applied, brings all balances to zero
AC-F	Data survives a page reload
AC-G	Corrupted stored data does not prevent the application from starting
AC-H	Exported data, when re-imported, reproduces the same state
AC-I	The application functions in the browsers listed in NFR-18
AC-J	The application is usable on a mobile viewport without horizontal scrolling

Requirements RF-01 to RF-03 in Section 3.14 are recommended, not mandatory, and are not required for acceptance.

---

10. LIMITATIONS

The limitations below follow directly from the architecture chosen. They are stated here so that the system is not represented as being more than it is.

10.1 Frontend-Only Architecture

All logic runs in the user's browser. There is no server to hold data centrally, to arbitrate between users, or to enforce rules independently. Every consequence in this section follows from this single fact.

10.2 localStorage Limitations

Limitation	Explanation
Device-bound	Data is stored by one browser on one device. Opening the application in a different browser, or on a different device, shows no data.
Origin-bound	Data is scoped to the origin from which the page was served. A page served from a different address does not see it.
Cleared with browsing data	Clearing site data, cookies or browsing history removes the stored data. Some browsers do this automatically under storage pressure.
Unavailable in some modes	Private browsing may block storage. The application handles this (FR-18) but cannot persist under it.
Finite quota	Browsers limit how much a single origin may store. The precise limit varies by browser and is not specified by any standard. The quantity of data this application produces is small relative to any such limit, but the limit exists.
Not backed up	There is no automatic backup. The only protection against loss is manual JSON export (FR-19).
Unencrypted	Data is stored as plain text and is readable through the browser's developer tools by anyone with access to the device.

10.3 No Multi-Device Synchronisation

Each installation is independent. If two members each open the application and record expenses, the two sets of data never meet. Transferring data between devices requires manual export and import.

10.4 No Authentication and No Security

The application has no login and no user accounts, because there is no server against which to authenticate.

It follows that the application provides **no security**. Anyone with access to the browser can view and modify the data. The privacy requirements of Section 4.6 guarantee only that data is not *transmitted* anywhere; they do not protect it on the device.

No claim of security is made, and none should be inferred. The application is suitable for informal expense-sharing among people who already trust one another, and is not suitable for data whose exposure would matter.

10.5 No Real Money Transactions

SettleUp calculates and displays what should be paid. It does not move money. Payments are made by the members themselves, through whatever means they use. The application has no way to know whether a payment has actually been made, and does not track this.

10.6 Other Limitations

Limitation	Explanation
Single group	This version manages one group at a time. Recording a second trip requires exporting the first and clearing the data.
Single currency	All amounts are treated as being in one currency. No conversion is performed.
No edit facility	An expense cannot be modified once recorded. Deletion is a **[Recommended]** feature (RF-01); editing is deferred to a future version.
No undo	No action can be reversed. Confirmation is required before destructive actions (NFR-05), but once confirmed they are final.
Greedy settlement is not provably minimal	See Section 7.11.
Concurrent use not handled	The application assumes one operator (A-03). Two browser tabs open simultaneously may overwrite one another's saved state.

---

11. FUTURE ENHANCEMENTS

The following are **not** part of the current project. They are recorded to show the direction in which the system could develop, and because several of them are made straightforward by decisions already taken.

Enhancement	Description	Enabled by
Edit an expense	Modify a recorded expense and recompute balances	Balances are already derived rather than stored (DD-01), so recomputation after an edit requires no additional mechanism
Rename a member	Change a member's name without affecting recorded expenses	Expenses reference identifiers, not names (Section 5.4.1), so the name can change freely
Multiple groups	Manage several independent groups	The state structure would gain a group level; the settlement logic is unchanged
Undo	Reverse a recent action	Would require an action history alongside the current state
Backend and database	Move storage to a server	Removes the device-bound and backup limitations of Section 10.2
User accounts	Authentication so that each member sees their own view	Requires a backend
Multi-device synchronisation	Members record expenses on their own devices, sharing one group	Requires a backend and conflict resolution
Multi-currency support	Record expenses in different currencies with conversion	Requires exchange rate data and a decision about which rate applies
Payment integration	Settle directly through a payment service	Requires a backend and a payment provider
Optimal settlement	Compute the true minimum number of payments for small groups by exhaustive search	Feasible for small n; infeasible in general (Section 7.11)
Recurring expenses	Automatically record regularly repeating costs such as rent	Extension of the existing expense model

---

12. CONCLUSION

SettleUp addresses a small but genuine problem: reconciling shared expenses within a group is tedious, error-prone, and typically results in more payments than are actually necessary.

The system computes each member's net position from the recorded expenses, and reduces the resulting obligations to a settlement of at most $n-1$ payments, in place of the $n(n-1)/2$ that a naive pairwise approach can produce. The reduction follows from a single observation — that only net positions matter, and that money passing through an intermediate member can be removed from the settlement entirely.

The application is built with HTML, CSS and vanilla JavaScript, without a framework, a backend, or a database. Data is stored in browser localStorage and can be exported as JSON. This architecture keeps the project within the scope of the

course, allows it to run without installation or an account, and keeps the group's data on the user's own device. The limitations it imposes are recorded in Section 10 without qualification.

For the author, the project serves a second purpose. Implementing form handling, validation, state management, DOM rendering, persistence and a non-trivial algorithm, without recourse to a library, exercises the material of the course directly rather than through an abstraction.

This document is a specification of what will be built. It will be revised if the requirements change during implementation, and the revision history at the front of the document will record any such change.

APPENDIX A — DETAILED DATA MODEL

A.1 Complete Structure

```
ApplicationState
├── members        : Member[]
├── expenses       : Expense[]
├── nextMemberId   : number
└── nextExpenseId  : number

Member
├── id             : number      unique, immutable
└── name           : string      non-empty; need not be unique

Expense
├── id             : number      unique, immutable
├── description    : string      non-empty
├── amount         : number      integer, paise, > 0
├── paidBy         : number      → Member.id
├── category       : string      non-empty
├── splitMethod    : string      "equal" | "exact" | "percentage"
├── shares         : Share[]     at least one entry
└── timestamp      : string      ISO 8601

Share
├── memberId       : number      → Member.id
└── amount         : number      integer, paise
```

A.2 Integrity Constraints

ID	Constraint
IC-01	`Member.id` is unique within `members`
IC-02	`Expense.id` is unique within `expenses`
IC-03	`Expense.paidBy` corresponds to an existing `Member.id`
IC-04	Every `Share.memberId` corresponds to an existing `Member.id`
IC-05	For each expense, the sum of `shares[].amount` equals `amount`
IC-06	Each expense has at least one share
IC-07	Within one expense, no `memberId` appears in more than one share
IC-08	No member referenced by any expense is absent from `members`

A.3 Worked Instance


```
{
  "members": [
    { "id": 1, "name": "Amit" },
    { "id": 2, "name": "Riya" },
    { "id": 3, "name": "Karan" }
  ],
  "expenses": [
    {
      "id": 1,
      "description": "Hotel",
      "amount": 600000,
      "paidBy": 1,
      "category": "Accommodation",
      "splitMethod": "equal",
      "shares": [
        { "memberId": 1, "amount": 200000 },
        { "memberId": 2, "amount": 200000 },
        { "memberId": 3, "amount": 200000 }
      ],
      "timestamp": "2026-08-14T10:30:00.000Z"
    },
    {
      "id": 2,
      "description": "Dinner",
      "amount": 150000,
      "paidBy": 2,
      "category": "Food",
      "splitMethod": "equal",
      "shares": [
        { "memberId": 1, "amount": 50000 },
        { "memberId": 2, "amount": 50000 },
        { "memberId": 3, "amount": 50000 }
      ],
      "timestamp": "2026-08-14T19:15:00.000Z"
    }
  ],
  "nextMemberId": 4,
  "nextExpenseId": 3
}
```


APPENDIX B — WORKED SETTLEMENT EXAMPLES

B.1 Chained Obligations

Balances: Amit −500, Riya 0, Karan +500

Riya has both paid and owed ₹500 across different expenses, leaving her net position at zero.

Iteration	Creditor	Debtor	Payment
1	Karan +500	Amit −500	Amit pays Karan ₹500

Result: 1 payment. Riya does not appear at all.

Had individual debts been settled rather than net positions, Amit would have paid Riya ₹500 and Riya would have paid Karan ₹500 — two payments, with money passing through Riya to no purpose.

B.2 One Member Pays for Everything

Setup: Five members. Amit pays ₹5,000, split equally among all five.

Balances: Amit +4,000; each of the other four −1,000

Iteration	Payment
1	Riya pays Amit ₹1,000
2	Karan pays Amit ₹1,000
3	Neha pays Amit ₹1,000
4	Vikram pays Amit ₹1,000

Result: 4 payments for 5 members. This is the n−1 bound exactly, and it is unavoidable here: four members each owe money and only one is owed any, so four transfers are necessary.

B.3 Six Members with Mixed Balances

Balances: A +4,500 · B +1,500 · C −500 · D −1,500 · E −2,000 · F −2,000

Sorted creditors: A (4,500), B (1,500) Sorted debtors: E (2,000), F (2,000), D (1,500), C (500)

Iteration	Creditor	Debtor	Amount	Payment
1	A +4,500	E −2,000	2,000	E pays A ₹2,000
2	A +2,500	F −2,000	2,000	F pays A ₹2,000
3	A +500	D −1,500	500	D pays A ₹500
4	B +1,500	D −1,000	1,000	D pays B ₹1,000
5	B +500	C −500	500	C pays B ₹500

Result: 5 payments for 6 members — the n−1 bound.

The naive pairwise approach on the same data could require up to 15 payments (6 × 5 ÷ 2). The reduction from 15 to 5 is the practical value of the algorithm.

B.4 Non-Divisible Amount

Setup: ₹100 (10,000 paise) split equally among three participants.

```
base      = floor(10000 / 3) = 3333 paise
remainder = 10000 − (3333 × 3) = 1 paisa

Participant 1:  3333 + 1 = 3334 paise  = ₹33.34
Participant 2:  3333          = ₹33.33
Participant 3:  3333          = ₹33.33
                ──────────────────────────
Total:          10000 paise   = ₹100.00  ✓
```

Rounding each share independently to ₹33.33 would total ₹99.99 and would leave one paisa unaccounted for, breaking BR-17.

APPENDIX C — FUNCTIONAL TEST CASES

ID	Precondition	Action	Expected result
TC-01	No members	Add "Amit"	Amit appears in the member list
TC-02	Amit exists	Add "Riya"	Both appear; Riya has a different identifier
TC-03	Amit exists	Add "Amit" again	Accepted; two members named Amit exist with different identifiers, and the interface distinguishes them
TC-04	Any state	Add an empty name	Rejected; no member created
TC-05	Any state	Add a name containing HTML markup	The name displays as literal text; no markup is rendered
TC-06	Amit has an expense	Remove Amit	Refused with an explanation
TC-07	Riya has no expense	Remove Riya	Riya is removed
TC-08	Members exist	Record an expense of 0	Rejected
TC-09	Members exist	Record an expense of −50	Rejected
TC-10	Three members	Record ₹300 equal among all three	Each share ₹100
TC-11	Three members	Record ₹100 equal among all three	Shares ₹33.34, ₹33.33, ₹33.33; total ₹100.00
TC-12	Three members	Record ₹1,200 exact: 500, 400, 300	Accepted; shares as entered
TC-13	Three members	Record ₹1,200 exact: 500, 400, 200	Rejected; shortfall of ₹100 shown
TC-14	Three members	Record ₹9,000 percentage: 50, 25, 25	Shares ₹4,500, ₹2,250, ₹2,250
TC-15	Three members	Record ₹9,000 percentage: 50, 25, 20	Rejected; running total of 95 shown
TC-16	Any set of expenses	Inspect balances	All balances sum to zero
TC-17	Amit paid ₹6,000 equal among three	Inspect Amit's balance	+₹4,000
TC-18	A member takes part in no expense	Inspect their balance	Zero
TC-19	A pays ₹300 split between B and C only	Inspect balances	A +300, B −150, C −150
TC-20	Amit +3,500, Riya −1,000, Karan −2,500	Generate settlement	Two payments, both to Amit
TC-21	All balances zero	Generate settlement	No payments; "settled" message
TC-22	Any settlement generated	Count payments	At most n−1
TC-23	Any settlement generated	Inspect amounts	No payment of zero
TC-24	Any settlement generated	Apply payments manually to balances	All balances become zero
TC-25	Chained obligations (Appendix B.1)	Generate settlement	One payment; intermediate member absent
TC-26	Single member group	Generate settlement	No payments
TC-27	Members and expenses recorded	Reload the page	All data present
TC-28	Data recorded	Inspect browser storage	Valid JSON matching Section 5.6
TC-29	Corrupt the stored value manually	Reload the page	Application loads and is usable, starting empty
TC-30	Storage blocked in browser settings	Open the application	Loads, accepts input, shows a non-retention notice
TC-31	Data recorded	Export	A valid JSON file downloads
TC-32	Exported file available	Import it	The same state is reproduced
TC-33	Prepare a file whose expense references a non-existent member	Import it	Rejected with a message; existing data unchanged
TC-34	Prepare a file that is not valid JSON	Import it	Rejected with a message; existing data unchanged
TC-35	Data recorded	Begin an import and cancel at the confirmation	Existing data unchanged
TC-36	Fresh application	View every screen	Each shows an explanatory empty state
TC-37	Expenses recorded	Search for a term present in one description	Only that expense is listed

ID	Precondition	Action	Expected result
TC-38	Expenses recorded	Apply a category filter with no matches	A "no results" message with a means of clearing filters
TC-39	Expenses in several categories	View category spending	Category totals sum to the group total
TC-40	Any state	View on a narrow mobile viewport	No horizontal scrolling; all controls reachable

APPENDIX D — CONSOLIDATED VALIDATION RULES

Field	Rule	On failure
Member name	Non-empty after trimming	"Please enter a name"
Member name	Duplicates are permitted; no uniqueness check	—
Member removal	Not referenced by any expense	"*Name* appears in recorded expenses and cannot be removed"
Expense description	Non-empty	"Please enter a description"
Expense amount	A number	"Please enter a valid amount"
Expense amount	Greater than zero	"Amount must be greater than zero"
Payer	One selected	"Please select who paid"
Participants	At least one selected	"Please select at least one participant"
Exact shares	Total equals the expense amount	"Shares total ₹X; ₹Y remains unaccounted for"
Exact shares	None negative	"Shares cannot be negative"
Percentages	Total exactly 100	"Percentages total X%; they must total 100%"
Percentages	None negative	"Percentages cannot be negative"
Import file	Parses as JSON	"This file is not valid JSON"
Import file	Contains the expected collections	"This file does not appear to be a SettleUp export"
Import file	All referenced members present	"This file refers to a member that is not in it"
Import	Confirmed by the user	Import does not proceed

APPENDIX E — GLOSSARY

Term	Meaning
Acceptance criteria	The conditions under which a requirement is judged to have been met
Business rule	A constraint that holds in the problem domain independently of the software
Creditor	A member whose net balance is positive; money is owed to them
Debtor	A member whose net balance is negative; they owe money
DOM	The browser's live representation of the page, which JavaScript can read and change
Edge case	An unusual input or condition at the boundary of what the system handles
Functional requirement	A statement of what the system does
Greedy algorithm	An algorithm that makes the locally best choice at each step and never revises it
IEEE 754	The floating-point standard that governs how JavaScript represents numbers
Invariant	A property that must hold at all times, such as balances summing to zero
JSON	A plain-text format for structured data, readable by both people and programs
localStorage	Browser storage that persists on the user's device across sessions
Net balance	Total paid minus total share, for one member
Non-functional requirement	A statement of a quality the system must possess
NP-hard	A class of problem for which no efficient general algorithm is known
Paise	The minor unit of the rupee; used internally to avoid floating-point error
Persistence	The property of data surviving after the application closes
Scope	The boundary of what a project includes and excludes
Settlement	A list of payments that clears all balances
Shall	In requirements writing, denotes an obligation
Share	One participant's portion of one expense
Traceability	The ability to follow a requirement back to the objective it serves
Vanilla JavaScript	JavaScript used without any framework or library
XSS	Cross-site scripting; an attack in which user input is executed as code

APPENDIX F — REQUIREMENTS TRACEABILITY MATRIX

Objective	Requirements	Tests
OBJ-01 — Remove manual arithmetic	FR-07, FR-08, FR-09, FR-10, FR-11; BR-10 to BR-18; DR-09	TC-10 to TC-19
OBJ-02 — Reduce payment count	FR-12; ALG in Section 7; BR-19 to BR-22	TC-20 to TC-26
OBJ-03 — No account, no connection, no installation	C-04, C-05, C-08; FR-16 to FR-18; NFR-21, NFR-22	TC-27 to TC-30
OBJ-04 — Data stays on the device	C-04, C-06; FR-16 to FR-20; NFR-21 to NFR-23	TC-27, TC-28, TC-31 to TC-34

Objective	Requirements	Tests
OBJ-05 — Demonstrate HTML, CSS and JavaScript competence	C-01, C-02, C-03; NFR-25 to NFR-28; Section 7	Whole of Appendix C

End of document.

Document status: Baseline, version 2.3 **Next review:** On completion of implementation, or on any change to the planned feature set